

Test di sistemi di assistenza medica alla diagnosi di malattie

Gruppo di lavoro

- Michele Pontrelli, 776836, m.pontrelli9@studenti.uniba.it

<https://github.com/MikiMik88/Progetto-ICON-25-26>

AA 2024-25

Introduzione

Il dominio d'interesse sono le malattie ed i sintomi ad esse associate. All'interno del dataset, sono presenti 41 malattie e 132 sintomi.

Fra i sintomi, uno di essi non è associato ad alcuna malattia, per tanto viene rimosso durante le operazioni di pre-processing del dataset.

Le malattie sono contenute tutte in una sola variabile categorica, per ogni entry nel dataset ad ogni malattia è associata una variabile booleana per ogni sintomo, essa è pari ad 1 se il sintomo è riscontrato nella malattia, 0 altrimenti.

Per ogni malattia ci sono esattamente 121 entry diverse i cui insiemi di sintomi variano l'uno dall'altro. Per tanto il dataset è perfettamente equilibrato.

Lo scopo è quello di testare diverse soluzioni sotto molteplici condizioni per capire quale di queste sia la più adatta per costruire un KBS che assista i medici nella diagnosi di malattie.

Sommario

Abbiamo diverse modalità con le quali svolgiamo il compito stabilito, nello specifico sono trattate: la rappresentazione mediante grafi, alcuni degli algoritmi di ricerca su grafi, l'apprendimento supervisionato attraverso 3 modelli compositi ed il ragionamento su modelli di conoscenza incerta mediante una rete bayesiana (o belief network).

Prima di passare il dataset ai vari moduli, esso subisce 2 operazioni di pre-processing, la prima, già menzionata, è la rimozione di un sintomo che non è causato da alcuna malattia; questo è fatto poiché la sua presenza genera problemi nell'allenamento di alcuni modelli e nel calcolo di alcune metriche. La seconda operazione di pre-processing è la trasformazione della variabile contenente le malattie da categorica a numerica, in particolare, ad ogni malattia è associato un valore numerico.

Elenco argomenti di interesse

- Rappresentazione con grafo e algoritmi di ricerca non informata (DFS e BFS) e informata (A^* e Branch and Bound) (Sezione 2)
- Apprendimento supervisionato con 3 diversi modelli compositi: SVM, AdaBoosting e GradientBoosting (Sezione 7)
- Ragionamento su modelli incerti di conoscenza mediante rete bayesiana (o belief network) (Sezione 9)

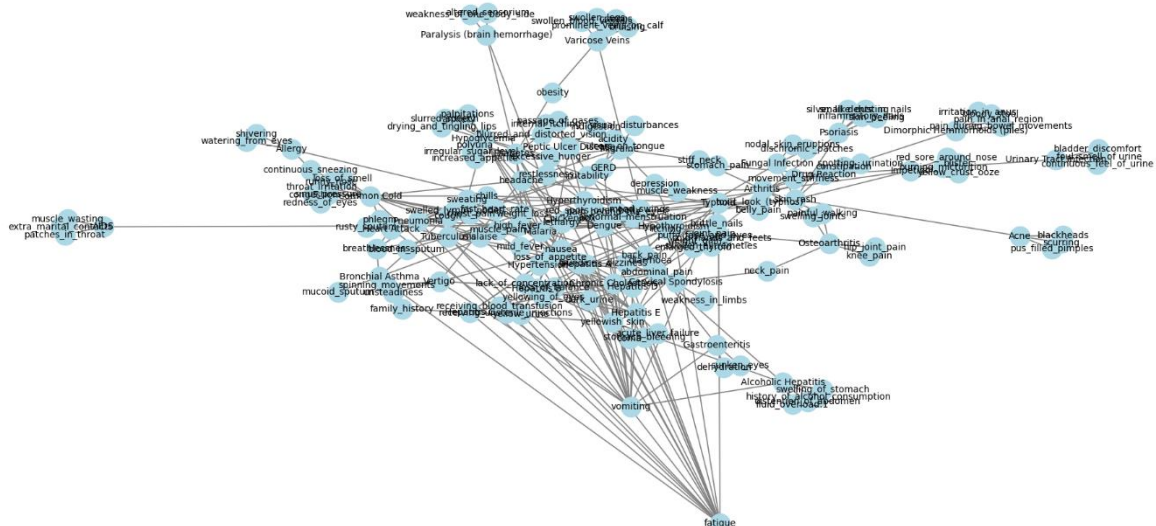
Sezione 1. Grafo e Algoritmi di Ricerca

Rappresentazione mediante grafo

Il dataset è stato rappresentato con un grafo pesato, dove ogni malattia ha archi provenienti dal sintomo verso la malattia. Il peso è dato dall'inverso della frequenza con cui i sintomi appaiono nel dataset, ossia un sintomo causato da più malattie ha peso minore rispetto a sintomi causati da meno malattie. In questa maniera nella ricerca euristica sintomi che portano a più diagnosi saranno preferiti.

Al peso dato agli archi è aggiunta una costante pari a 0.01, così da non avere archi con pesi nulli, cosicché gli algoritmi di ricerca informata rimangano ammissibili.

Si allega a seguito una visualizzazione, sfortunatamente un po' confusionaria del grafo risultante (senza visualizzazione dei pesi):



Alcune informazioni interessanti sul grafo:

- il grafo ha 172 nodi e 321 archi;
- la malattia con più sintomi associati è: Common Cold (Raffreddore comune) con 17 sintomi;
- la malattia con meno sintomi associati è: Fungal Infection (Infezione funginea) con 4 sintomi;

Anche se un po' difficile da notare, si possono scorgere nel grafico alcuni esempi di sintomi legati ad una specifica malattia, in particolare si noti il nodo AIDS all'estrema sinistra che porta con sé 3 sintomi causati solo da esso.

Algoritmi di ricerca usati e descrizione dell'euristica

Come anticipato, sono stati utilizzati 4 algoritmi di ricerca su grafo, 2 non informati, ossia Deep First Search (DFS) e Breadth First Search (BFS), e 2 informati, ossia A* e Branch and Bound.

Al fine di testare gli algoritmi sul grafo, sono stati generati 10 insiemi di sintomi contenenti ciascuno dall'uno ai 5 sintomi in maniera randomica. Ogni insieme è stato dato a tutti gli algoritmi di ricerca,

per ognuno di essi si è tenuto traccia delle performance attraverso le metriche che descriverò a breve.

Si noti che la ricerca anziché partire da un nodo iniziale parte da molteplici, questo è fatto semplicemente inserendo tutti i nodi dei sintomi nell'insieme da subito nella frontiera, simulando la presenza di un nodo di partenza che si colleghi ad essi.

Le implementazioni di DFS, BFS, A* e B&B sono standard.

L'euristica per gli algoritmi di ricerca informata funziona nel seguente modo: calcola il numero di sintomi "mancanti" per raggiungere la malattia target. In pratica, prende l'insieme dei sintomi associati alla malattia target (cioè, i nodi collegati alla malattia) e lo confronta con l'insieme dei sintomi collegati al nodo corrente. Il risultato, cioè il numero di elementi mancanti, viene usato come stima del costo residuo. L'euristica è ammissibile perché non sovrastima il costo reale, ma non considera i pesi degli archi.

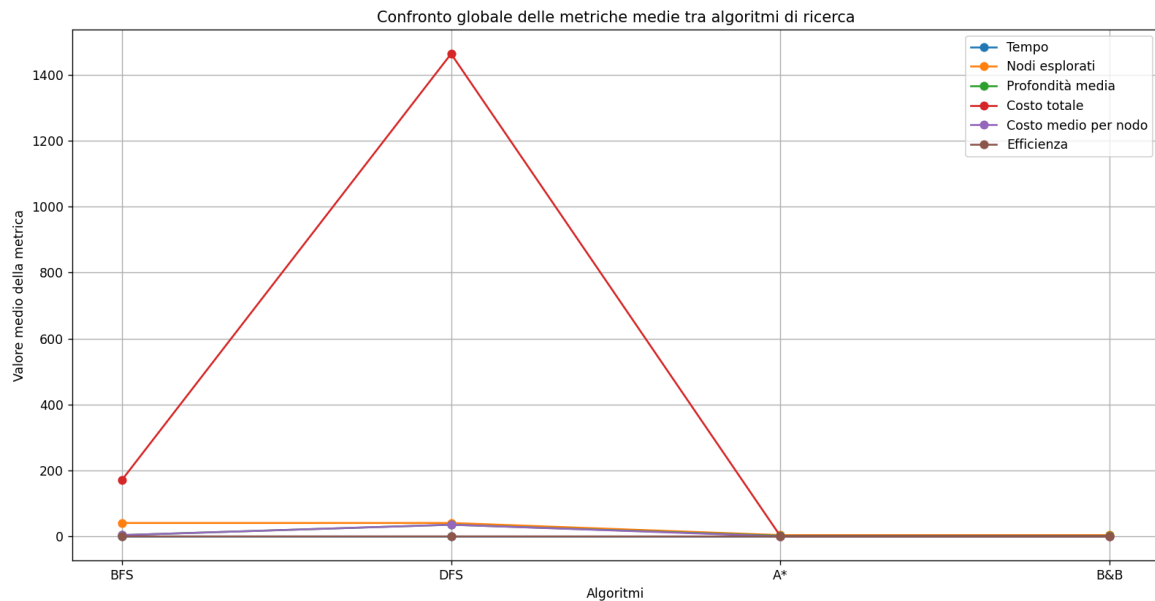
Descrizione delle metriche di valutazione usate

Le metriche di valutazione utilizzate sono le seguenti:

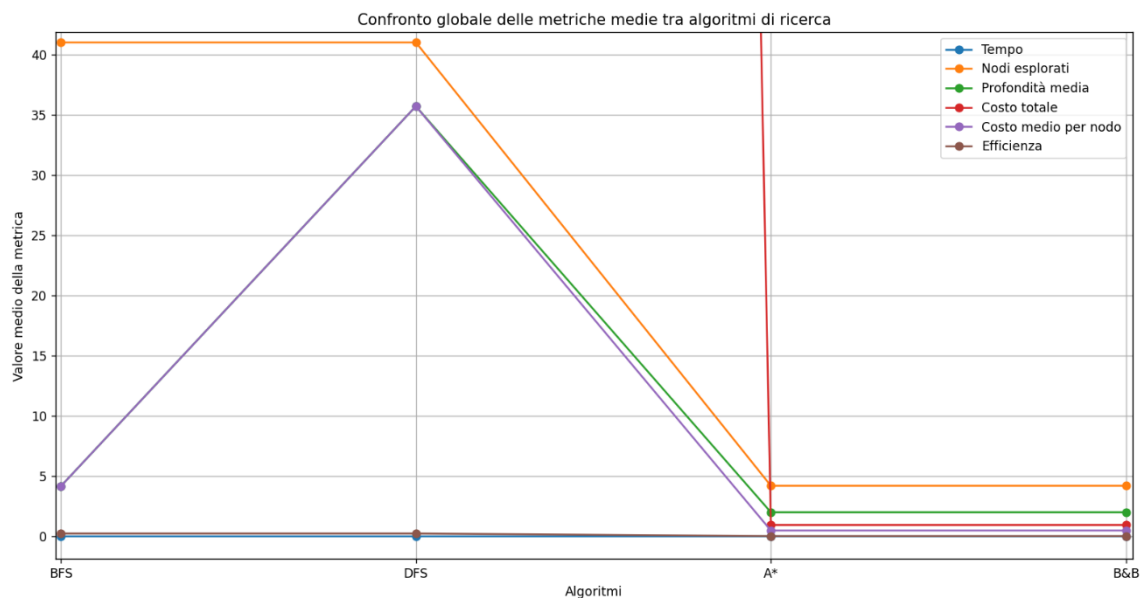
- Tempo: il tempo, in secondi, impiegato dall'algoritmo per trovare i percorsi.
- Nodi esplorati: il numero di soluzioni trovate, ossia il numero di percorsi che terminano in un nodo malattia.
- Profondità media: la lunghezza media dei percorsi trovati.
- Costo Totale: la somma dei costi dei percorsi (si noti che per DFS e BFS corrisponde al numero di visite, mentre per A* e B&B corrisponde alla somma dei pesi dei nodi visitati).
- Costo medio per nodo: il costo totale diviso il numero di soluzioni trovate.
- Efficienza: rapporto tra il numero di nodi esplorati e il numero totale di nodi nel grafo. Più è basso, più la ricerca è mirata.

Analisi dei risultati ottenuti

A seguito dei test, abbiamo ottenuto i seguenti risultati, riportati nel grafico qui sotto:



Ingrandendo sui valori più piccoli:



Si può notare come la metrica del tempo sia praticamente pari a 0 per ogni algoritmo, questo è dato dal fatto che lo spazio di ricerca è comunque molto limitato per tanto nessuno di essi ha difficoltà a terminare velocemente.

Per quanto riguarda l'efficienza essa è leggermente superiore per DFS e BFS, significando che hanno performato peggio rispetto ad A* e B&B. Considerato come questa metrica è calcolata, ciò è dato dal fatto che gli algoritmi di ricerca non informata hanno esplorato più nodi prima di terminare rispetto agli altri 2.

La profondità media è uguale per A* e B&B, leggermente superiore per BFS e molto più elevata per DFS, ma questo c'era da aspettarselo se consideriamo il funzionamento di DFS.

Il costo medio per nodo segue lo stesso trend della profondità media, per ragioni affini.

Il costo totale è comparabile unicamente fra DFS e BFS e A* e B&B dal momento in cui sono calcolati in maniera differente fra algoritmi di ricerca informata e non. Possiamo notare come BFS abbia effettuato circa 200 visite, mentre DFS oltre le 1400, ma questo c'è da aspettarselo, se ancora una volta, consideriamo il funzionamento dell'algoritmo DFS. Per quando riguarda A* e B&B invece, hanno praticamente stesso costo totale.

Conclusioni

Ne concludiamo che A* e B&B in questo caso hanno performance praticamente identiche e molto buone secondo queste metriche, mentre DFS e BFS si differenziano fra loro e rispetto agli algoritmi di ricerca informata, evidenziando come essi si muovano parecchio all'interno del grafo prima di trovare una soluzione e terminare; si può considerare una prova sperimentale degli effetti della funzione euristica negli algoritmi di ricerca.

Sezione 2. Apprendimento Supervisionato

Sommario

L'apprendimento supervisionato è stato fatto, come detto, su 3 modelli SVM, Adaptive Boosting e Gradient Boosting. Per ogni modello è stata utilizzata la Cross Validation K-Fold con $K = 5$ e metriche log-loss, accuracy, precision e recall; la cross validation minimizza la log-loss ed è stata ripetuta per 3 volte per ogni configurazione di iperparametri testata, cosicché i risultati ottenuti non fossero dipendenti da una specifica suddivisione del dataset ma bensì la media di 3 suddivisioni differenti.

Per SVM e Gradient Boosting, indipendentemente dal valore fornito agli iperparametri, le metriche misuravano sempre prestazioni praticamente perfette, nonostante la presenza della cross validation per limitare il fenomeno dell'overfitting. Per tanto ho deciso di testare la robustezza di questi due modelli introducendo vari livelli di rumore nel dataset e attraverso la selezione di un numero inferiore di features di input.

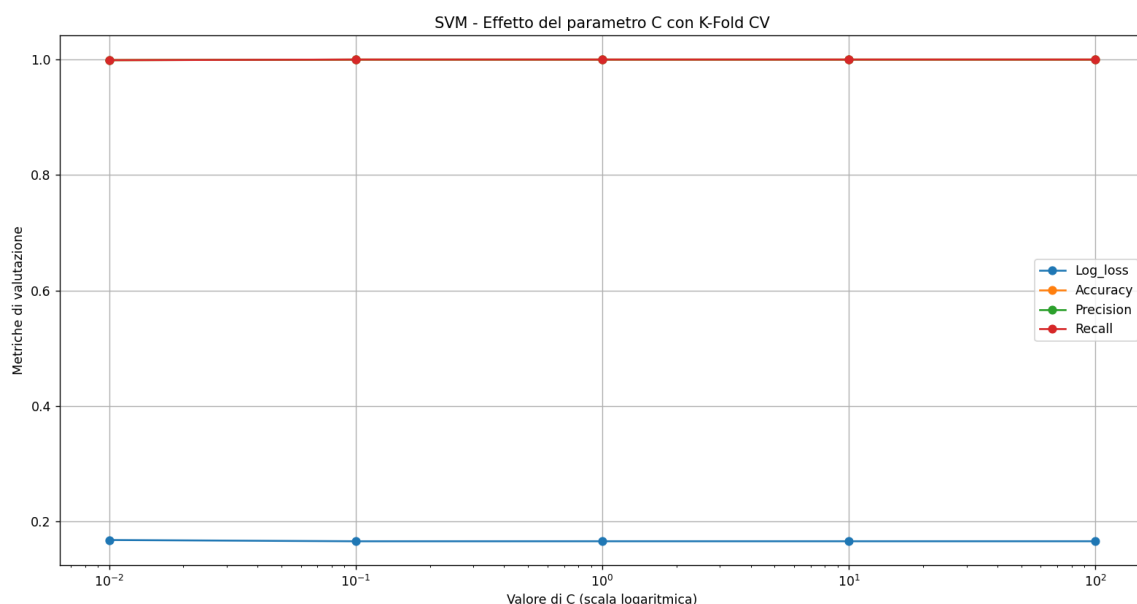
SVM

Test con iperparametri

Per l'SVM è stato utilizzato un kernel lineare ed i test sono stati effettuati su di un unico iperparametro a diversi livelli: C . Esso ha una funzione di regolarizzazione, in particolare, la regolarizzazione sui dati è inversamente proporzionale al suo valore, per tanto un C basso implica una forte regolarizzazione, un C alto una debole.

I valori di C su cui sono stati effettuati i test sono i seguenti: $[0.01, 0.1, 1, 10, 100]$

Il risultato dei test è riportato in questo grafico:



Analisi dei risultati

Si noti come al variare di C tutte le metriche utilizzate (log loss per la cross validation, accuracy, precision e recall) mantengono lo stesso valore, nel caso della log loss ~ 0.17 e nel caso delle altre 3 metriche ~ 1.00 (nel grafico Accuracy, Precision e Recall sono sovrapposte).

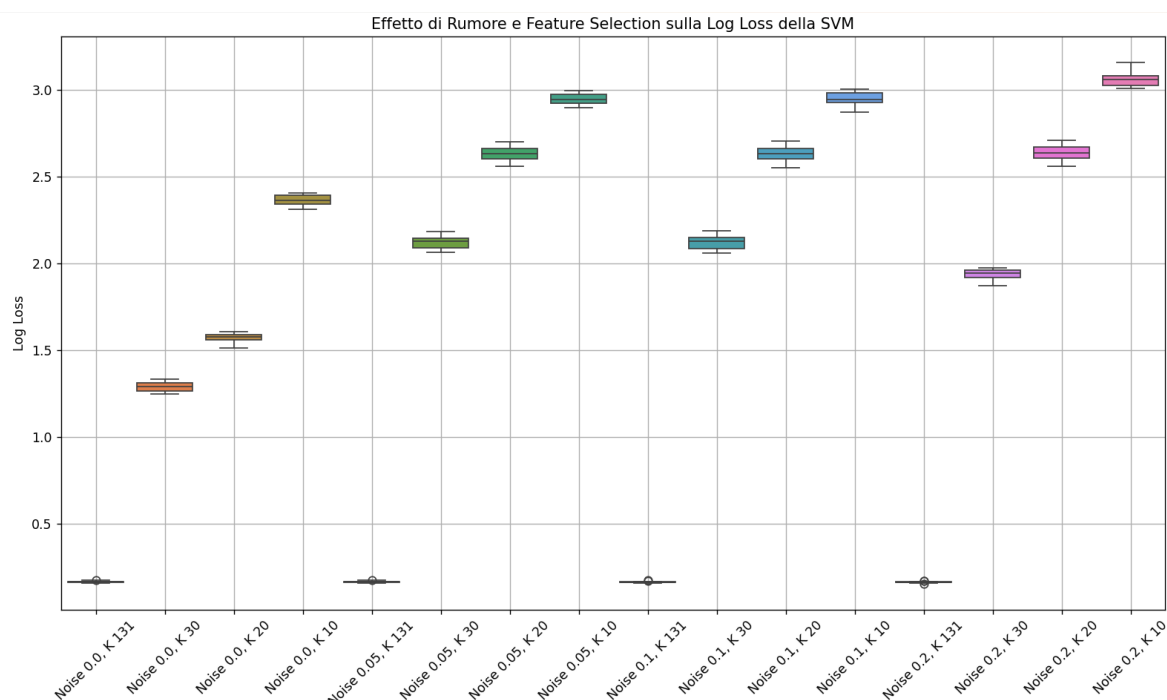
Riporto inoltre un aneddoto interessante in merito alla ripetizione della cross validation: ad un certo punto dello sviluppo non era ancora stata introdotta la ripetizione, per tanto veniva effettuata ogni volta una suddivisione specifica del dataset in 5 parti. Per via di questa specifica suddivisione, la log loss per i valori di C 0.01 e 0.1 era più alta di quanto non lo fosse in realtà. A seguito dell'aggiunta della ripetizione, la valutazione effettiva è stata misurata.

Test con rumore e feature selection

A seguito di ciò, ho quindi effettuato i test con introduzione del rumore sui dati e feature selection, in particolare i valori utilizzati sono stati i seguenti:

- Rumore: [0.0, 0.05, 0.1, 0.2]
- Features considerate: [131(tutte), 30, 20, 10]

Sono stati effettuati test per ogni combinazione di valori di rumore e feature selection, per ogni test è stata applicata la cross validation k-fold con $k = 5$ ripetuta 3 volte. Sono stati ottenuti i seguenti risultati, riassunti in questa tabella:



In questo caso guardiamo unicamente alla log loss per rendere più chiaro il grafico.

Analisi dei risultati

Per ogni combinazione di valori è indicata la log loss corrispondente, in particolare, si può notare nel grafico la presenza di una linea nera che passa attraverso la parte colorata, quella è la log loss media, le barre nere al di sopra e al di sotto della parte colorata sono la deviazione standard rispetto alle 3 cross validation effettuate.

Si può notare come, selezionando tutte le feature, indipendentemente dal livello di rumore aggiunto, il modello performi al massimo delle sue capacità.

Passando alla selezione di sole 30 features, anche senza alcun rumore aggiunto, abbiamo subito una impennata della log loss, che indica una prestazione inferiore del modello.

Si può notare come alla riduzione delle feature selezionate e all'aumento del rumore introdotto, le performance del modello peggiorino di pari passo. L'unica eccezione la abbiamo con 30 features selezionate e rumore pari a 0.2, dove il modello performa leggermente meglio rispetto a quando è selezionato lo stesso numero di features ma con rumore inferiore.

Per quanto riguarda le altre metriche, nel caso con log loss maggiore hanno i seguenti valori:

- Accuracy: Media=0.1685, Dev.Std=0.0139
- Precision: Media=0.2506, Dev.Std=0.0341
- Recall: Media=0.1707, Dev.Std=0.0056

Nel caso con log loss minore:

- Accuracy: Media=1.0000, Dev.Std=0.0000
- Precision: Media=1.0000, Dev.Std=0.0000
- Recall: Media=1.0000, Dev.Std=0.0000

Considerando un caso intermedio, ad esempio rumore = 0.1 e feature selezionate = 30:

- Accuracy: Media=0.3871, Dev.Std=0.0121
- Precision: Media=0.4042, Dev.Std=0.0180
- Recall: Media=0.3900, Dev.Std=0.0054

Si nota quindi come anche le altre metriche seguano lo stesso pattern della log loss, ovvero denotano performance inferiori all'aumentare di rumore e riduzione di numero di feature selezionate.

Adaptive Boosting

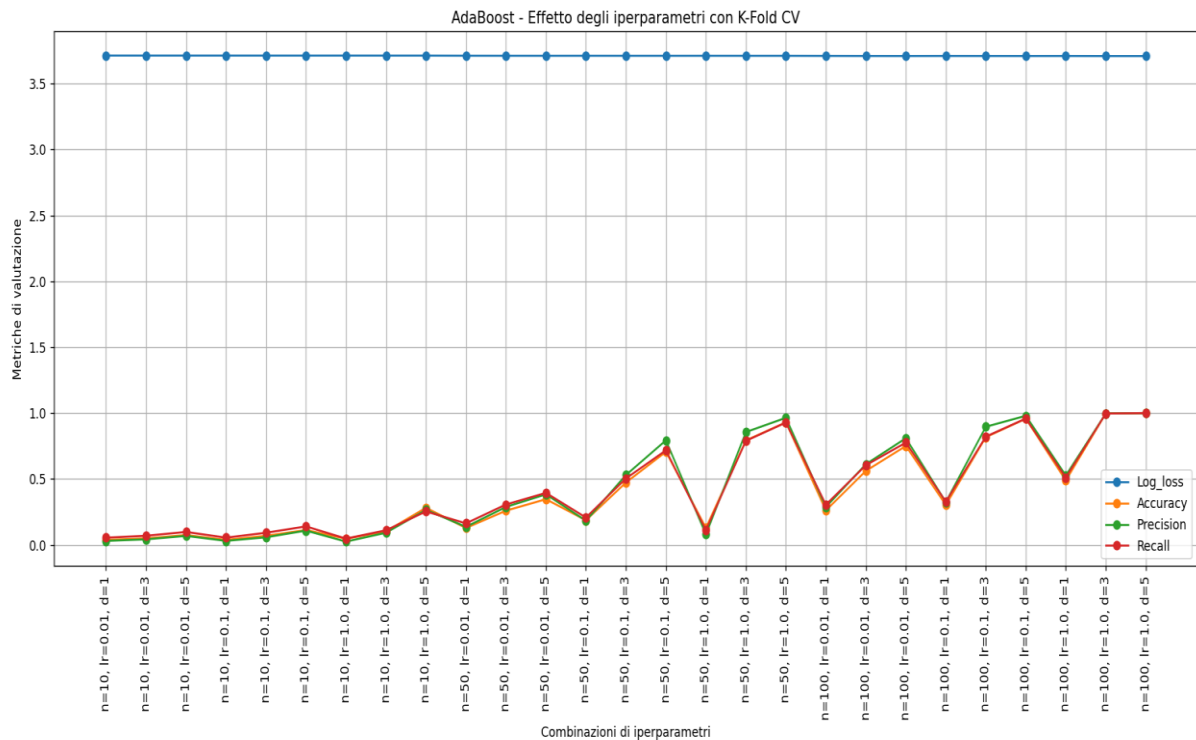
Test sugli iperparametri

Nell'adaptive boosting gli iperparametri considerati sono stati il numero di stimatori, il learning rate e gli stimatori veri e propri; in particolare sono stati utilizzati i seguenti valori:

- Numero stimatori: [10, 50, 100]
- Learning Rate: [0.01, 0.1, 1.0]
- Stimatori: Albero di Decisione con profondità massima 1 (Stump), 3 e 5

Dove lo stimatore è lo stimatore di base da cui l'ensemble è composta, il numero di stimatori è il massimo di stimatori ammessi nell'ensemble (in caso di perfect fit s'interrompe prima di raggiungere il numero massimo) e il learning rate è l'importanza, espressa mediante il peso, che ogni stimatore ha nel modello.

Sono stati quindi testate tutte le combinazioni di iperparametri, risultanti nel seguente grafico:



Analisi dei risultati

Si può notare che avere lo stump come stimatore di base non dia mai grandi risultati, indipendentemente da quanti se ne usino e da quanta importanza gli venga data.

Discorso diverso lo abbiamo invece per gli alberi con profondità 3 e 5, i quali riescono molto più facilmente ad arrivare a prestazioni elevate all'aumentare del numero massimo e dell'importanza.

Notiamo che in generale all'aumentare del numero di massimo di stimatori e dell'importanza di ognuno di essi, aumentano le prestazioni del modello.

L'eccezione ai trend descritti è la log loss, la quale rimane sempre fissa a ~3.7, mentre le altre metriche riescono ad arrivare anche a prestazioni perfette.

Come anticipato, dal momento in cui in questo caso non ci sono stati risultati perfetti indipendentemente dai valori degli iperparametri, ho preferito concentrarmi su di essi, come è stato visto sopra, senza testare anche la robustezza del modello.

Gradient Boosting

Test sugli iperparametri

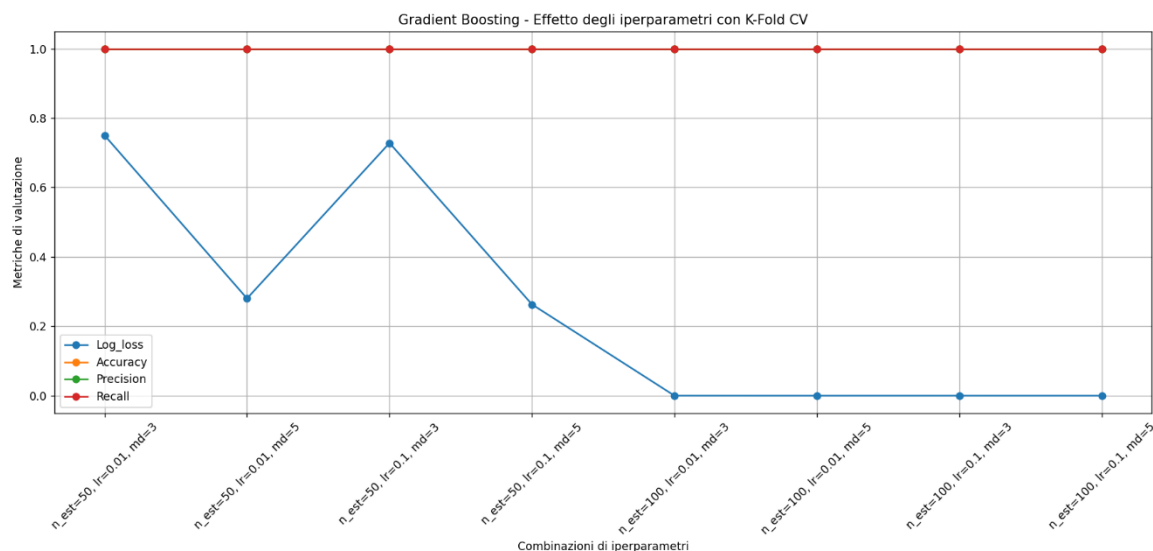
In questo modello gli iperparametri individuati sono abbastanza simili a quelli dell'adaptive boosting:

- Numero di stimatori: [50, 100]
- Learning rate: [0.01, 0.1]
- Profondità massima: [3, 5]

Si nota innanzitutto la presenza di 2 valori per ogni iperparametro, questo è dovuto al fatto che i tempi per questo modello erano troppo estesi per poter testare 27 combinazioni diverse di parametri e per ognuna di esse effettuare la ripetizione della cross validation per 3 volte (arrivando a 405 iterazioni).

In ogni caso, in questo modello il numero di stimatori ed il learning rate hanno lo stesso significato che avevano nell'adaptive boosting, ossia il numero di stimatori di base che formano l'ensemble e l'importanza di ognuno di essi. Il terzo parametro è invece direttamente la profondità massima anziché un albero con una specifica profondità massima dal momento in cui nel gradient boosting si usano come stimatori di base solo alberi di regressione.

Per le diverse combinazioni di iperparametri si sono ottenuti i seguenti risultati:



Analisi dei risultati

Come evidente e come anticipato, anche in questo modello abbiamo avuto performance perfette per quasi tutte le metriche indipendentemente dai valori degli iperparametri, tranne che per la log loss che di fatto diminuisce all'aumentare del numero di estimatori, del learning rate e della profondità massima degli alberi.

Test con rumore e feature selection

Allora anche in questo caso ho testato la robustezza del modello attraverso l'introduzione di vari livelli di rumore nel dataset e la selezione di features. Sempre per i lunghi tempi di apprendimento del modello, anche in questo caso (rispetto all'SVM) il numero di valori per rumore e feature selection sono stati ridotti da 4 a 3 e sono i seguenti:

- Rumore: [0.0, 0.1, 0.2]
- Features selezionate = [131(tutte), 30, 10]

Sono stati rimossi casi più intermedi (0.5 per il livello di rumore e 20 per il numero di feature selezionate) anziché i valori massimi dal momento in cui ciò ci permette di valutare gli effetti di rumore e feature selection anche singolarmente anziché solo come combinazione.

Analisi dei risultati

Sfortunatamente ci sarebbe voluto così tanto tempo che il testing per le combinazioni coi valori più elevati di rumore e feature, in particolare rumore 0.2 e feature 30 e 10, non è stato completato in tempo, per tanto sono presenti soltanto i dati per le prime 7 combinazioni in forma testuale, anziché in grafico:

Rumore: 0.0, Feature selezionate: 131:

- Log_loss: Media=0.0000, Dev.Std=0.0000
- Accuracy: Media=1.0000, Dev.Std=0.0000
- Precision: Media=1.0000, Dev.Std=0.0000
- Recall: Media=1.0000, Dev.Std=0.0000

Rumore: 0.1, Feature selezionate: 131:

- Log_loss: Media=1.1851, Dev.Std=0.0309
- Accuracy: Media=0.5431, Dev.Std=0.0126
- Precision: Media=0.9130, Dev.Std=0.0121
- Recall: Media=0.5557, Dev.Std=0.0039

Rumore: 0.2, Feature selezionate: 131:

- Log_loss: Media=2.2982, Dev.Std=0.0348
- Accuracy: Media=0.3288, Dev.Std=0.0103
- Precision: Media=0.9736, Dev.Std=0.0011
- Recall: Media=0.3374, Dev.Std=0.0018

Rumore: 0.0, Feature selezionate: 30:

- Log_loss: Media=2.3594, Dev.Std=0.0473
- Accuracy: Media=0.3853, Dev.Std=0.0117
- Precision: Media=0.3883, Dev.Std=0.0056
- Recall: Media=0.3861, Dev.Std=0.0060

Rumore: 0.0, Feature selezionate: 10:

- Log_loss: Media=2.3622, Dev.Std=0.0400
- Accuracy: Media=0.3894, Dev.Std=0.0112
- Precision: Media=0.3927, Dev.Std=0.0036
- Recall: Media=0.3899, Dev.Std=0.0038

Rumore: 0.1, Feature selezionate: 30:

- Log_loss: Media=3.3205, Dev.Std=0.0555
- Accuracy: Media=0.1920, Dev.Std=0.0133
- Precision: Media=0.1922, Dev.Std=0.0048
- Recall: Media=0.1925, Dev.Std=0.0048

Rumore: 0.1, Feature selezionate: 10:

- Log_loss: Media=3.4569, Dev.Std=0.0478
- Accuracy: Media=0.1695, Dev.Std=0.0103
- Precision: Media=0.1699, Dev.Std=0.0037
- Recall: Media=0.1698, Dev.Std=0.0037

Analisi dei risultati

I dati risultanti sono stati ordinati per log loss crescente. Si può notare come i livelli più bassi li abbiamo con la selezione di tutte le feature, indipendentemente dal livello di rumore introdotto nel dataset. A differenza dell'SVM però, la log loss non rimane a 0 ma aumenta, indicando come questo modello sia meno robusto di SVM.

Per quanto riguarda le altre combinazioni, la log loss aumenta all'aumentare del rumore introdotto e al diminuire delle features selezionate.

Le altre metriche seguono per la maggior parte lo stesso trend con alcune eccezioni.

Sezione 3. Ragionamento con rete bayesiana

Sommario

Infine, ho utilizzato una rete bayesiana per calcolare la malattia più probabile a partire dai set di sintomi. Sono stati effettuati test scegliendo diversi livelli di numeri di features. Sfortunatamente non sono stato in grado di fare inferenza in maniera appropriata sulla rete bayesiana, per tanto verranno riportati soltanto i risultati ottenuti sull'insieme di test.

Problemi e possibili soluzioni

Ogni qualvolta provare a fare inferenza su insiemi di sintomi generati casualmente a partire da quelli presenti nella rete, è sempre risultato in un errore causato da un eccesso di memoria. Per questa ragione ho provato a ridurre il numero delle features considerate, scendendo fino a 30, ma nulla. Inoltre arrivato a quei livelli, come si vedrà nella parte dedicata alla valutazione, le performance della rete erano troppo basse per essere considerata valida, per cui anche se avesse funzionato, non avrebbe prodotto buoni risultati

Ho provato anche ad ottimizzare la struttura della rete mediante HillClimbing, che permette l'introduzione di variabili latenti, ma non è stato testato a fondo e non ha prodotto risultati rilevanti.

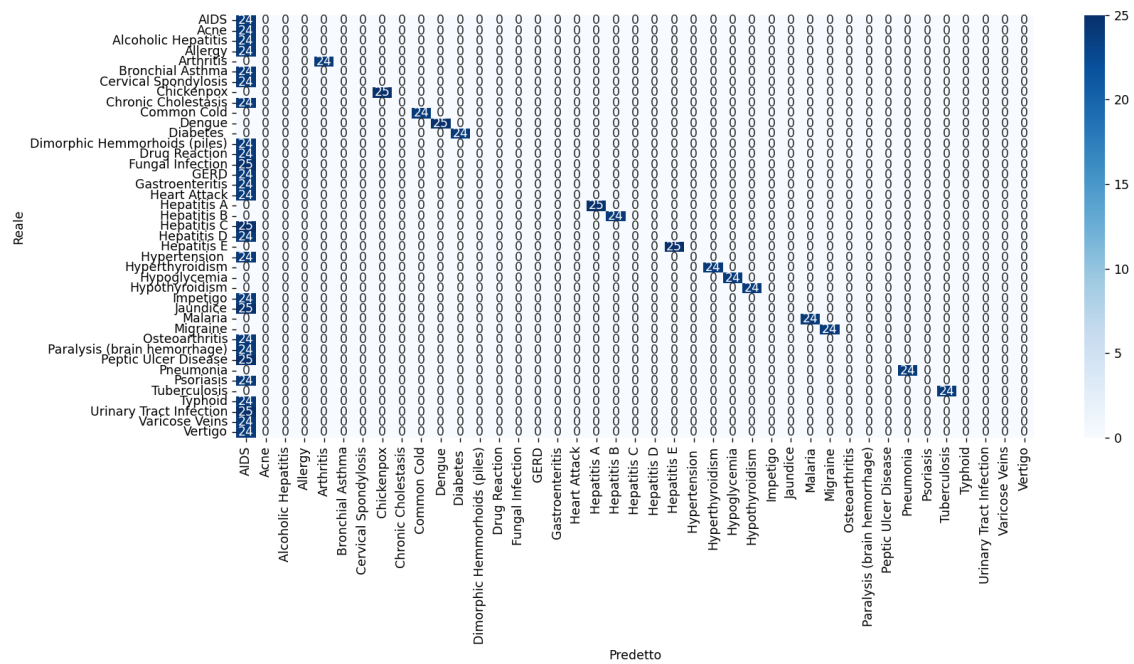
Un'altra possibile soluzione era l'introduzione di feature sintetiche, ovvero: nel dataset sono presenti sintomi associati ad una singola malattia e malattie che presentano come sintomi unicamente questa tipologia di sintomi. Essendo questi poco informativi, poiché collegati ad una sola malattia, essi vengono scartati quando sono selezionate le migliori feature. L'idea era quella di raggruppare queste tipologie speciali di sintomi in un'unica feature per ogni malattia che li presentasse, ma, come detto, l'idea non è stata applicata per mancanza di tempo.

Valutazione

La rete è stata testata con 30, 50, 80 e 131 features. All'aumento del numero di features si è avuto un miglioramento in termini di accuratezza.

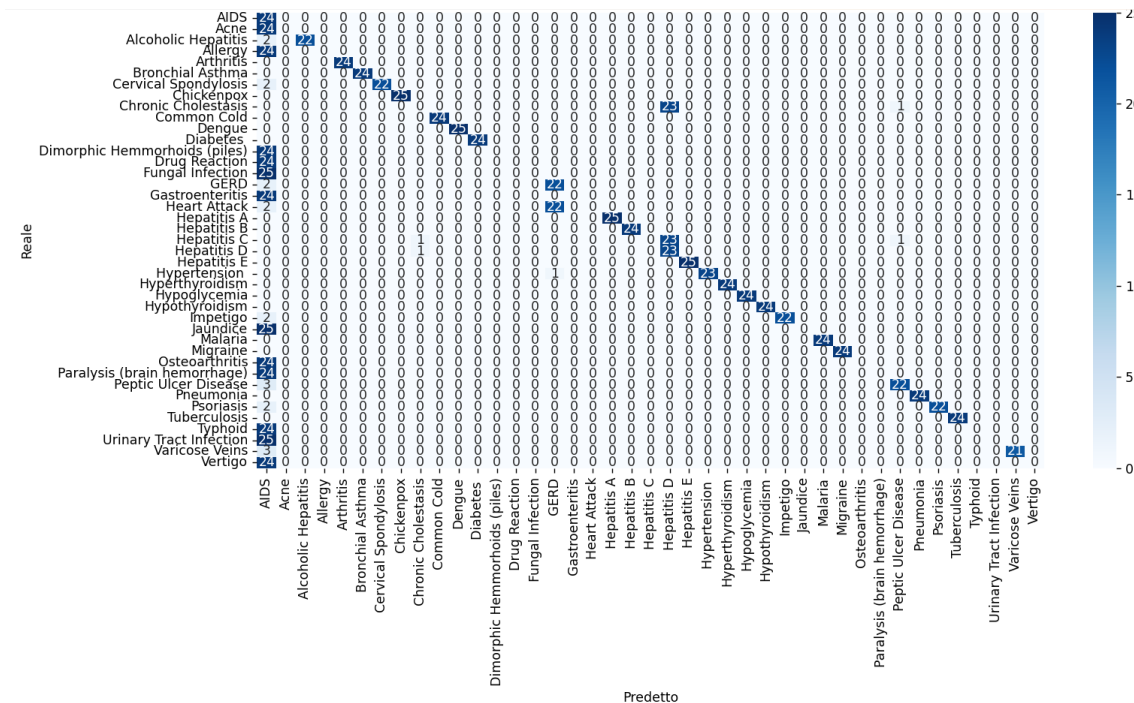
In particolare, mostrerò per ogni livello, l'accuratezza ottenuta sull'insieme di test ed una matrice che mostra, data una malattia, quante volte essa sia stata predetta a sé stessa o ad un'altra malattia.

Partendo da 30 features, l'accuratezza sul test set ottenuta è circa 0.39, mentre la matrice è la seguente:

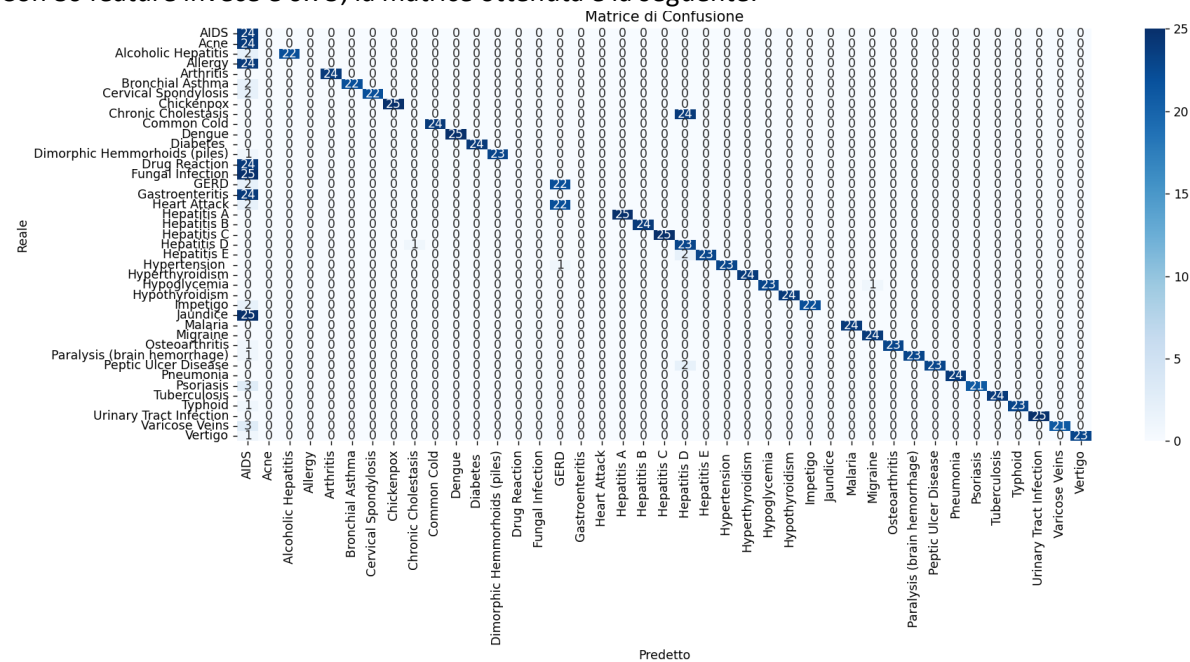


Si può notare come poche malattie siano effettivamente predette a sé stesse, la maggior parte sono state considerate AIDS, unicamente poiché questa è la prima malattia nella lista, indicando che la probabilità che gli insiemi di sintomi nell'insieme di test avessero stessa probabilità per tutte le malattie presenti.

Con 50 feature l'accuratezza ottenuta è 0.61, mentre la matrice è la seguente:



Con 80 feature invece è 0.75, la matrice ottenuta è la seguente:



Ed infine con 131 feature la rete ha ottenuto risultati praticamente perfetti.,

In conclusione, all'aumento delle feature considerate, come prevedibile, si è avuto un aumento nelle prestazioni della rete.

Conclusioni

Grafi e algoritmi di ricerca

Va notato come gli algoritmi spesso non concordassero sulla malattia, in particolare:

- A* e B&B arrivavano sempre alla stessa conclusione
- DFS a volte concordava con gli algoritmi di ricerca informata, altre volte con BFS e altre volte ancora (ma più raramente) si discostava da tutti e tre
- BFS non concordava quasi mai con gli algoritmi di ricerca informata

Per tanto mi sarebbe piaciuto trovare un modo per capire quale delle diagnosi ottenute per ogni insieme di sintomi fosse la più corretta.

Inoltre, avrei voluto testare più euristiche e valutarne l'impatto sugli algoritmi di ricerca.

Infine, avrei voluto trovare delle metriche più uniformi per confrontare gli algoritmi.

Apprendimento

Per SVM, avrei voluto testare anche altri kernel, oltre a quello lineare, e capire come questo possa impattare sulle sue performance.

Per Adaptive Boosting sarebbe stato interessante testare anche la robustezza con i parametri che hanno performato meglio. Inoltre, avrei voluto capire la ragione per cui la log loss rimane fissa a circa 3.7 indipendentemente dagli iperparametri.

Per Gradient Boosting, infine, avrei voluto testare almeno sullo stesso numero di valori che ho usato su SVM.

Ragionamento con rete bayesiana

Mi sarebbe piaciuto avere più tempo per sperimentare maggiormente le idee discusse nella sezione relativa ed effettuare inferenza sulla rete bayesiana, al fine di capirne le capacità nell'ambito e poterla valutare e confrontare con le altre soluzioni viste.

Valutazione finale

Fra le varie tecniche viste, pare che la più funzionale in termini di performance, tempi di apprendimento e robustezza sia di fatto SVM con Kernel lineare, per tanto in un futuro, sarebbe interessante vedere come performa su un dataset più realistico, ossia con molte più malattie e sintomi e probabilmente non equilibrato come quello usato in questo caso di studio.